Beginner's Guide to Automating Tasks with Bash

Bash scripting is your superpower for turning boring, repetitive terminal commands into one-click automated tasks. Mastering this saves you hours of manual work and prevents human typos during late-night deployments.

What you'll learn:

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Storing Information with Variables

Variables act like named boxes where you can store data like file names, user inputs, or server addresses. Instead of typing the same value over and over, you assign it once at the top of your script. You reference your variable by putting a dollar sign in front of its name. This makes your scripts clean, flexible, and easy to update later.

```
$SERVER_IP="192.168.1.50"
 echo "Connecting to $SERVER_IP..."
```

2

TIP 2 OF 4

Making Decisions with If-Else

Computers are great at making choices if you give them clear rules using conditional statements. You can check if a file exists, if a server is reachable, or if a user has admin permissions. If the condition is true, the computer runs path A; otherwise, it runs path B. This stops your scripts from crashing blindly when unexpected things happen.

```
if [ -f "config.json" ]; then
  echo "Config found! Starting app..."
else
  echo "Error: Missing config file."
fi
```

3

TIP 3 OF 4

Repeating Tasks with Loops

Why type a command fifty times for fifty different files when a loop can do it in a second? A for loop lets you iterate through a list of items and run the same command on every single one. It is the ultimate tool for processing bulk logs, renaming multiple images, or checking health statuses across a fleet of servers.

```
$for filename in report.txt data.txt logs.txt; do
   echo "Processing $filename..."
 done
```

4

TIP 4 OF 4

Keeping Track with Exit Codes

Every time a command finishes running in Linux, it silently returns a number called an exit code. A code of zero means everything went successfully, while any other number means an error occurred. You can check this code right after a risky command to decide whether your script should keep running or stop immediately.

```
$apt-get update
 if [ $? -eq 0 ]; then
   echo "Update successful!"
 else
   echo "Update failed. Check your network."
 fi
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always add #!/bin/bash at the very top of your script files.
+ Make your script executable using the chmod +x command before running it.
+ Use descriptive names for your variables instead of single letters.
+ Test your commands line-by-line in the terminal before putting them in a script.
+ Add comment notes explaining WHY you wrote tricky sections of code.

Watch Out For

! Putting spaces around the equals sign when defining variables (e.g., VAR = value will fail).
! Forgetting to quote your variables, which causes errors if filenames have spaces.
! Running untested delete or move commands on important production folders.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh